C Programming

- C = Middle level language & High level language both & General purpose language **(used for many types of tasks)**
  (C को कभी कभी middle level language इसलिए कहते हैं, क्योंकी इसमें pointer, direct memory access जैसे feature use करते हैं एवं High level इसलिए कहते हैं क्युकी इसमें हम English जैसे word - if,while,printf,.....use करते हैं |)
- C is a Procedural Oriented Programming Language (POP)
- Designed / Developed by = **Dennis Ritchie**
- Year = 1972 **at Bell Laboratories**
- C is not Object Oriented (No OOP concept)
- Based on = Structured / Procedural Programming **(smallest unit = function)**
- Approach / follow = Top to Down **OR** Top to Bottom
  (Top to Down means पहले बड़ा problem देखो फिर छोटा problem देखों)
- C → Platform dependent source code **(needs recompilation for different systems)**
- Application / Used in = System Software, Operating System, Embedded Systems, Compilers, Device Drivers, etc.
- Extension of C = .c
- C is the base / mother language of many languages (C++, Java, etc.)
- C is very important for beginners in programming
- C is one of the most widely used programming languages
- C uses Compiler (GCC, clang, etc.)

- C is also called = Middle Level Language **(because it supports both low-level and high-level features)**
- C is a case sensitive language.
- Used Compiler/Source code where C Run i.e. IDE (Integrated Development environment)
  - ➔ Turbo C++ / Borland C++
  - ➔ Code :: Block
  - ➔ Dev c++ (Developer C++)
  - ➔ Visual Studio Code (Microsoft)

    GCC (GNU Compiler Collection)

• What is IDE ?

→Integrated Development Enviroment (एक तरह का tool box हैं, जिसमें debug,compiler,loder,linker,.....होते हैं)

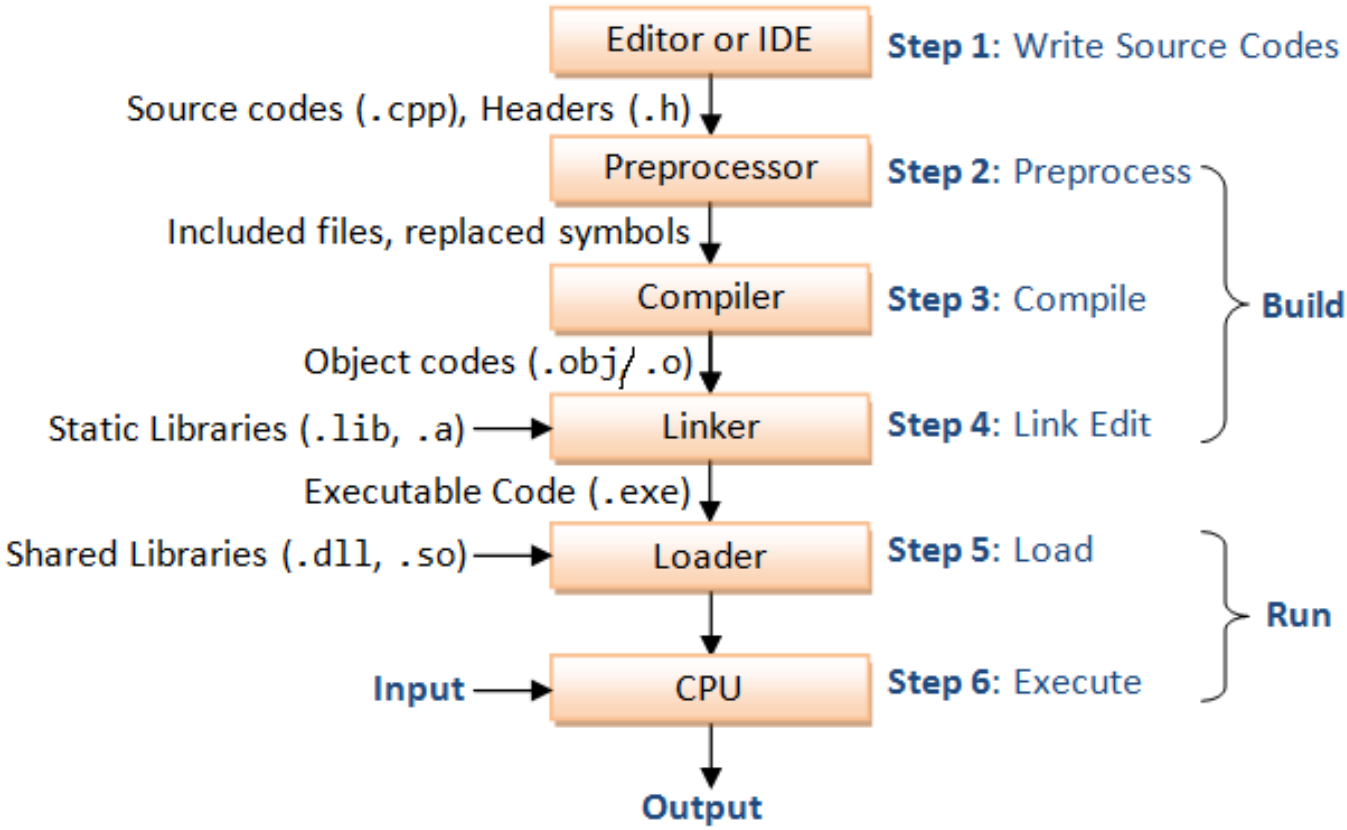
IDE Consist of-

i. Source Code Editor
ii. Build Automation Tool
iii. Debugger
iv. Compiler/ Interpreter

Difference Between C and C++

C	C++
1.C is Procedural Language.	1. C++ is non Procedural i.e Object oriented Language.
2. No virtual Functions are present in C	2. The concept of virtual Functions are used in C++.
3. In C, Polymorphism is not possible.	3. The concept of polymorphism is used in C++. Polymorphism is the most Important Feature of OOPS.
4. Operator overloading is not possible in C.	4. Operator overloading is one of the greatest Feature of C++.
5. Top down approach is used in Program Design.	5. Bottom up approach adopted in Program Design.
6. No namespace Feature is present in C Language.	6. Namespace Feature is present in C++ foravoiding Name collision.

❖ C Flow Diagram



Step 1: Preprocessing / Preprocessor

The first step in the compilation process is preprocessing. This step is where the compiler processes the source code and performs various tasks such as **including header files, expanding macros, and removing comments.** The result of this step is a modified version of the source code, which is then passed on to the next step of the compilation process.

Step 2: Compilation (Compiler)

The next step in the compilation process is compilation. This step is where the preprocessed **source code is compiled into object code(.obj /.o) or Machine code**. Object code is a low-level representation of the code that can be executed by a computer.

During the compilation step, the compiler reads the preprocessed source code and performs the following tasks: **Syntax analysis** (grammatically correct)**, Code generation** (generates object code from the source code)**, Semantic analysis** (checking for variable declarations, function definitions, and other language-specific features). If any error then show **Compiler error**.

Step 3: Linking / Linker

The final step in the compilation process is linking. This step is where the object files are linked together to form an executable program. The linker performs the following tasks: Symbol resolution, Relocation, and Output generation.

Object file + required libraries = Executable/binary file (.exe file → Windows)

Step 4: Optimization

After the executable file is generated, it can be further optimized to improve its performance. Optimization is the process of improving the efficiency of the compiled code by making it execute faster, use less memory, or both. Optimization can be done at various levels, including the compiler, linker, and operating system.

Step 5: Debugging

The Next step in the C++ compilation process is debugging. Debugging is the process of identifying and fixing errors in the compiled code. Debugging can be done using various tools, including debuggers, profilers, and memory checkers.

Step 6: Execution (Run Program)

Execute .exe file by the CPU.

❖ Library Function

A library function in C is a pre-written, pre-compiled function that is part of the C Standard Library or other external libraries. (ऐसा लाइब्रेरी जिसमें पहले से ही लिखा होता हैं कि इस function का क्या काम हैं |)

e.g. = printf(), scanf(), sqrt(), pow(), strlen(),……

❖ Header File

C header files are files with a **.h** extension (library function को use करने के लिए हम header file का use करते हैं)

e.g. = #include<stdio.h>, #include <cmath.h>,…….

❖ Some Library function & header function with works/purpose

Header File	Purpose	Example Functions
<stdio>	Standared Input/output	Printf(), scanf()
<string>	String handling	length(), substr(), find()
<vector>	Dynamic arrays	push_back(), size(), at()
<algorithm>	Algorithms	sort(), reverse(), find()
<cmath>	Math functions	sqrt(), pow(), sin(), abs()
<cstdlib>	General utilities	rand(), malloc(), exit()
<ctime>	Time manipulation	time(), clock(), difftime()
<map>	Sorted associative containers	insert(), find(), erase()
<set>	Set containers	insert(), find(), count()
<queue>	Queue containers	push(), pop(), front()
<stack>	Stack containers	push(), pop(), top()
<list>	Doubly linked list	push_back(), remove(), sort()
<array>	Fixed-size array	at(), size()

Header File	Purpose	Example Functions
<bitset>	Bit manipulation	set(), reset(), test()
<functional>	Function objects	std::function, bind, mem_fn
<thread>	Multithreading	thread, join, detach
<mutex>	Synchronization primitives	mutex, lock_guard
<chrono>	Time measurement	steady_clock, duration
<fstream>	File handling	ifstream, ofstream, open()
<sstream>	String stream	stringstream, getline()
<numeric>	Numeric operations	accumulate(), iota()
<regex>	Regular expressions	regex_match(), regex_search()
<conio>	Take any key input and clrscr	Clrscr() , getch()

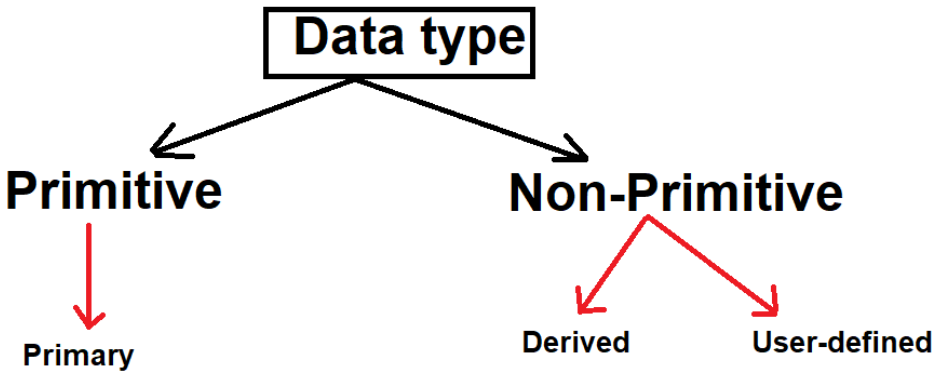
- scanf → take input from user
- printf → print output
- \n → change line / new line
- // → Single line comment
- /*……*/ → Multi line comment

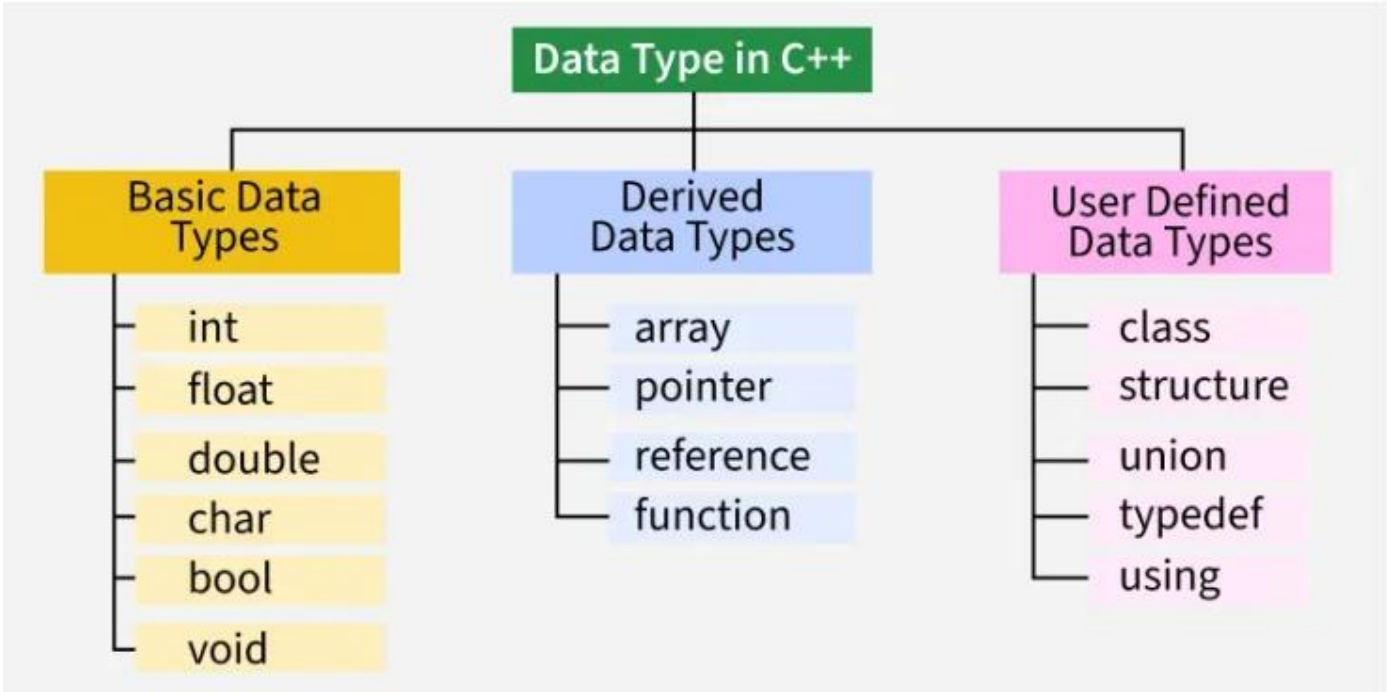
❖ Data Type

- The data type determines the kind of information that may be stored in the variable. Variables are classified according to their data type.
- Whenever a variable is defined in C++, the compiler allocates memory for that variable based on the data type with which it is declared.
- The programmer can select the data type appropriate to the needs of the application. Data types specify the size and types of values to be stored.

C++ supports the following data types:

1. Primary or Built-in or Fundamental/Basic data type
2. Derived data types
3. User-defined data types





❖ Size of Primitive Data type

(64 bit compilers – 2025-2026)

Data Type	Size (byte)	Use	Range
int	4	To store integers numbers	-2147483648 to 2147483647
long	4	For large integers numbers	-2147483648 to 2147483647
long long	8	For very large integers numbers	-9223372036854775808 to 9223372036854775807
float	4	For decimal numbers	-3.4 * 10^38 to3.4*10^38
double	8	For high precision decimal numbers	+- 1.7 * 10^308
long double	8	For extra high precision decimal numbers (some compiler 12/16)	+- 1.1 * 10^4932
char	1	To store a single character	'A', 'a', '0', …..
bool	1	To store true/false values	True/false

Old 16bit /Turbo C++ sizes

Type	Size (bits)	Size (bytes)	Range
char	8	1	-128 to 127
unsigned char	8	1	0 to 255
int	16	2	-2^{15} to $2^{15}-1$
unsigned int	16	2	0 to $2^{16}-1$
short int	8	1	-128 to 127
unsigned short int	8	1	0 to 255
long int	32	4	-2^{31} to $2^{31}-1$
unsigned long int	32	4	0 to $2^{32}-1$
float	32	4	3.4E-38 to 3.4E+38
double	64	8	1.7E-308 to 1.7E+308
long double	80	10	3.4E-4932 to 1.1E+4932

HIRA KUMAR

❖ Format Specifiers in C

In C language, format specifiers are used in input and output functions (such as printf() and scanf()) to define the type of data being handled.

Specifier	Type	Description
%d	int	Decimal integer
%c	char	Single character
%s	char[]	String of characters
%f	float	Floating-point number
%lf	double	Double precision floating-point number
%u	unsigned int	Unsigned decimal integer
%x	unsigned int	Hexadecimal integer
%o	unsigned int	Octal integer
%p	void*	Pointer address
%e, %E	float, double	Scientific notation
%g, %G	float, double	Use %e or %f based on value

==========HIRA KUMAR==========